HARDHAT + ETHERS.JS

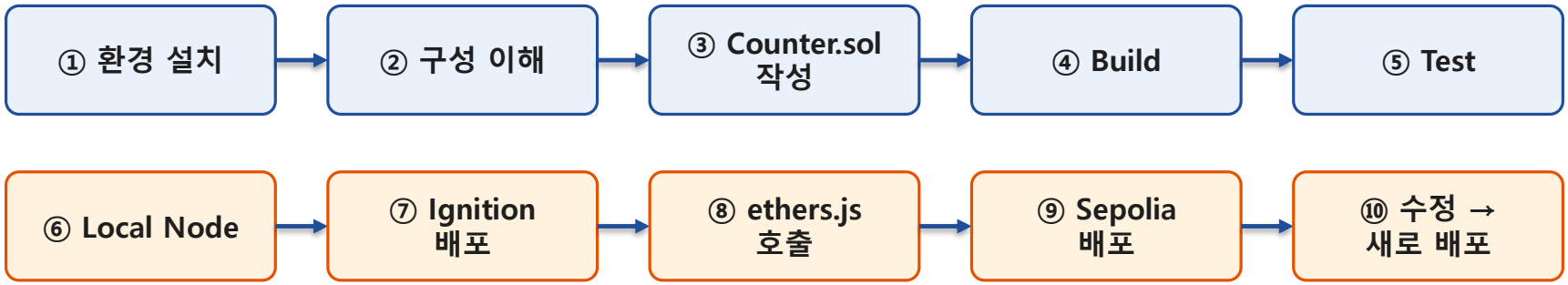
인하공업전문대학 컴퓨터정보공학과
최효현 교수

주요사항

- 개발 환경
  - ✓ Hardhat 3, ethers.js 소개
  - ✓ Node.js, npm, VS Code, Starter
  - ✓ package.json, hardhat.config.ts
- 작성, Build
  - ✓ Counter.sol
  - ✓ Build: Bytecode, ABI
- 부록
  - ✓ 테스트가 필요한 이유, Mocha + Chai 테스트 작성
- 배포와 호출
  - ✓ Hardhat Local Node
  - ✓ Ignition 배포
  - ✓ ethers.js: Provider, Signer, Contract
- Sepolia 배포
  - ✓ keystore, 배포, Etherscan 확인
- Contract 수정과 재배포
  - ✓ 불변성, --reset, underflow 확인
- 컨트랙트 추가하기 (Storage)
- 자주 나는 오류와 해결

Hardhat 3

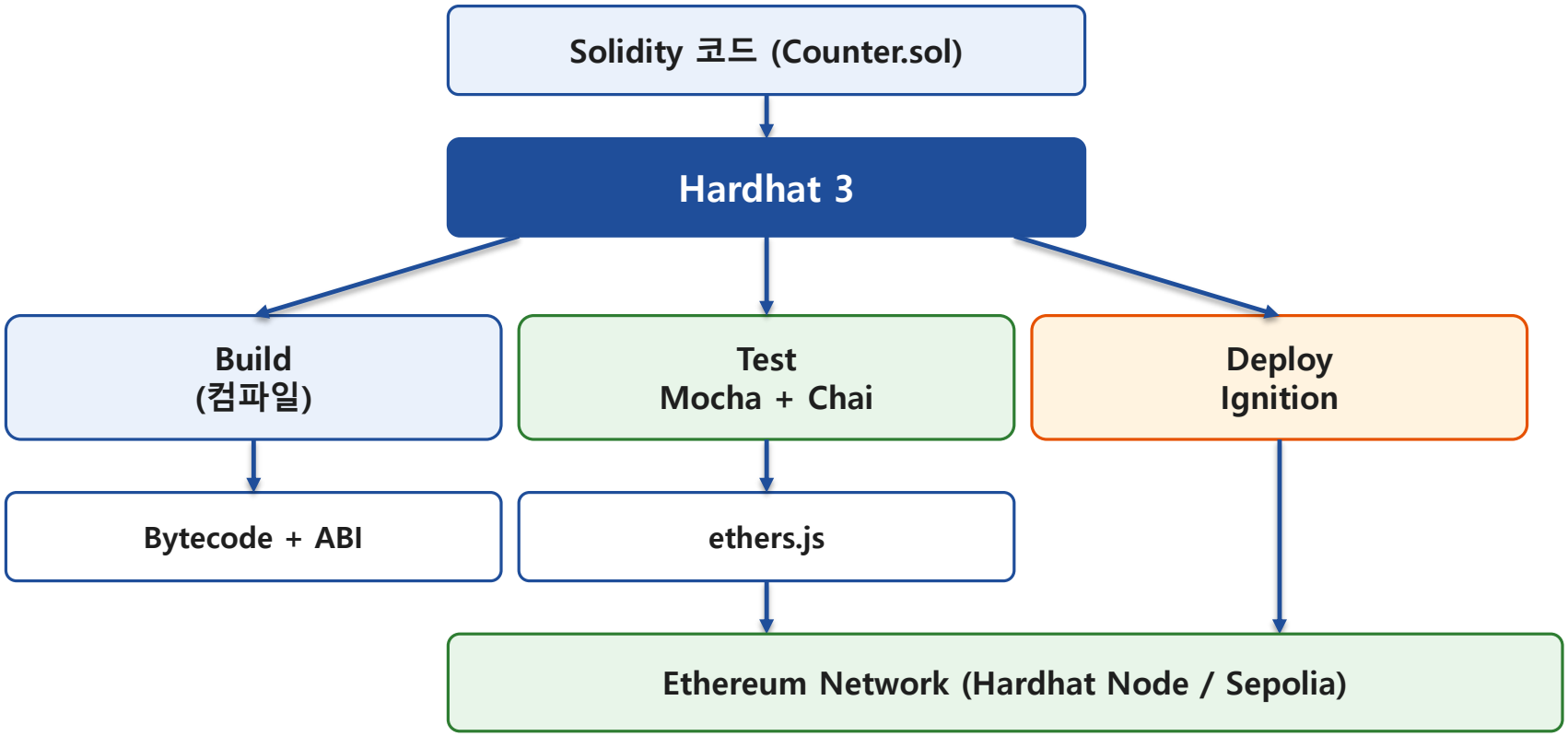
❑ 수업 목표: 스마트 컨트랙트 개발의 전체 흐름을 한 번 경험하기



- ✓ Remix에서 버튼으로 하던 일을 코드와 명령으로 직접 수행
- ✓ 실제 개발 현장의 방식: 자동 테스트, 배포 기록 관리, JavaScript 프로그램과 연결
- ✓ 핵심 3가지 개념: **Provider → Signer → Contract** (ethers.js)

Hardhat 3

❑ 전체 구조



Hardhat 3

- 왜 Hardhat + ethers.js 인가?
  - ✓ Remix: 웹 브라우저에서 바로 실습하기 좋음, 큰 프로젝트 및 자동화에는 불편
  - ✓ Hardhat: 내 PC에서 코드와 명령으로 개발 → 테스트 자동화, 버전 관리 편리
  - ✓ ethers.js: JavaScript 프로그램에서 Ethereum과 통신하는 대표 라이브러리

- 참고
  - ✓ Hardhat 공식 문서의 기본 예제는 viem + node:test 조합
  - ✓ 우리 수업은 ethers.js + Hardhat 조합 (Mocha 테스트는 나중에 함)

Hardhat 3

❑ Remix와 Hardhat 비교: 같은 일을 다른 방법으로

하는 일	Remix	Hardhat 3 + ethers.js
코드 작성	File Explorer	VS Code (contracts/Counter.sol)
컴파일	Compile 버튼	npx hardhat build
테스트	직접 버튼 눌러 확인	npx hardhat test (자동 테스트)
테스트용 블록체인	Remix VM	Hardhat Node (npx hardhat node)
배포	Deploy 버튼	Ignition (npx hardhat ignition deploy)
함수 호출	주황 / 파랑 버튼	ethers.js 코드 (counter.increment())
Sepolia 연결	MetaMask (Injected Provider)	RPC URL + 개인 키 (keystore)
ABI 확인	Compile 탭의 ABI 복사	artifacts/.../Counter.json

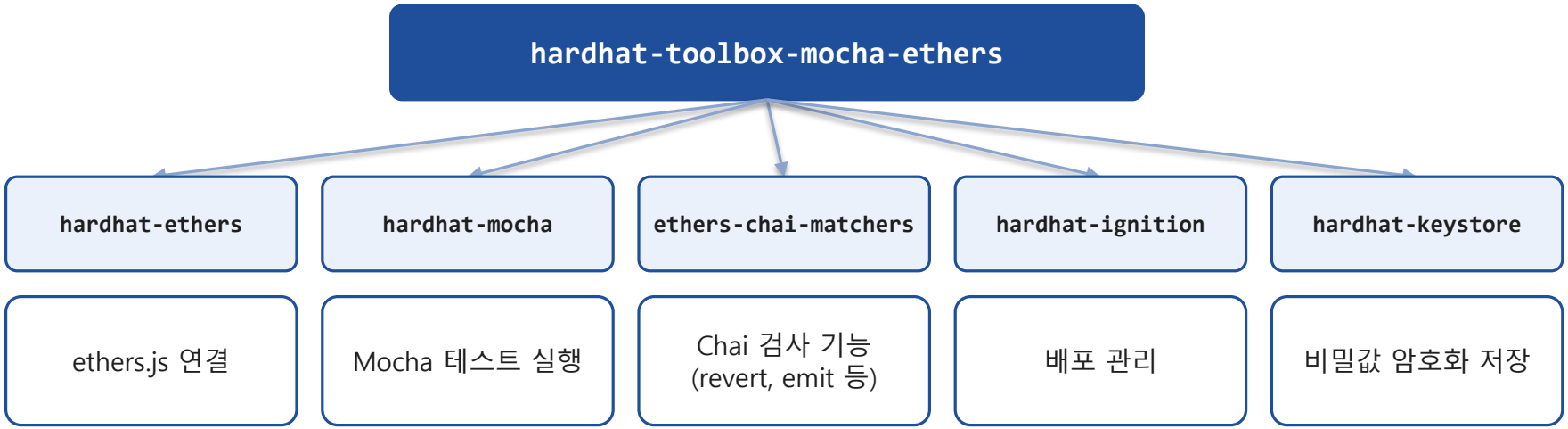
개발 환경

❑ 처음에 알아야 할 구성 요소

요소	쉬운 설명
Node.js	JavaScript / TypeScript 프로그램을 PC에서 실행
npm	필요한 JavaScript 패키지를 설치하는 도구
package.json	프로젝트에서 사용할 패키지 목록
Hardhat 3	Solidity 개발, 컴파일, 테스트, 배포 환경
Plugin	Hardhat에 기능을 추가하는 확장 모듈
Toolbox	함께 쓰는 여러 Plugin을 묶어 놓은 패키지
ethers.js	JavaScript에서 Ethereum과 통신하는 라이브러리
Mocha	테스트 코드를 실행하는 도구 (describe, it)
Chai	결과가 맞는지 검사하는 도구 (expect)
Ignition	스마트 컨트랙트 배포 관리 도구
Hardhat Node	내 PC에서 실행하는 테스트용 Ethereum 네트워크
Sepolia	공개 Ethereum 테스트넷 (Token, NFT 실습에서 사용)

개발 환경

- **Toolbox 구조: 플러그인 묶음 하나로 필요한 기능을 한 번에 설치**



- ✓ **Mocha, Chai는 나중에 사용할 것임**
- ✓ **package.json에는 Toolbox 하나만 적어도, 위 플러그인과 Mocha, Chai가 함께 설치됨**

개발 환경

❑ 설치 준비 Node.js

- ✓ Hardhat 3은 Node.js 22.13 이상 필요 (LTS 버전 권장)
- ✓ VS Code 터미널(PowerShell)에서 확인 → 없거나 낮으면 설치, 업데이트

① 버전 확인

```
PS C:\> node -v
v24.20.0
PS C:\> npm -v
11.19.0
```

② 설치, 업데이트 (winget: Windows 기본 설치 도구)

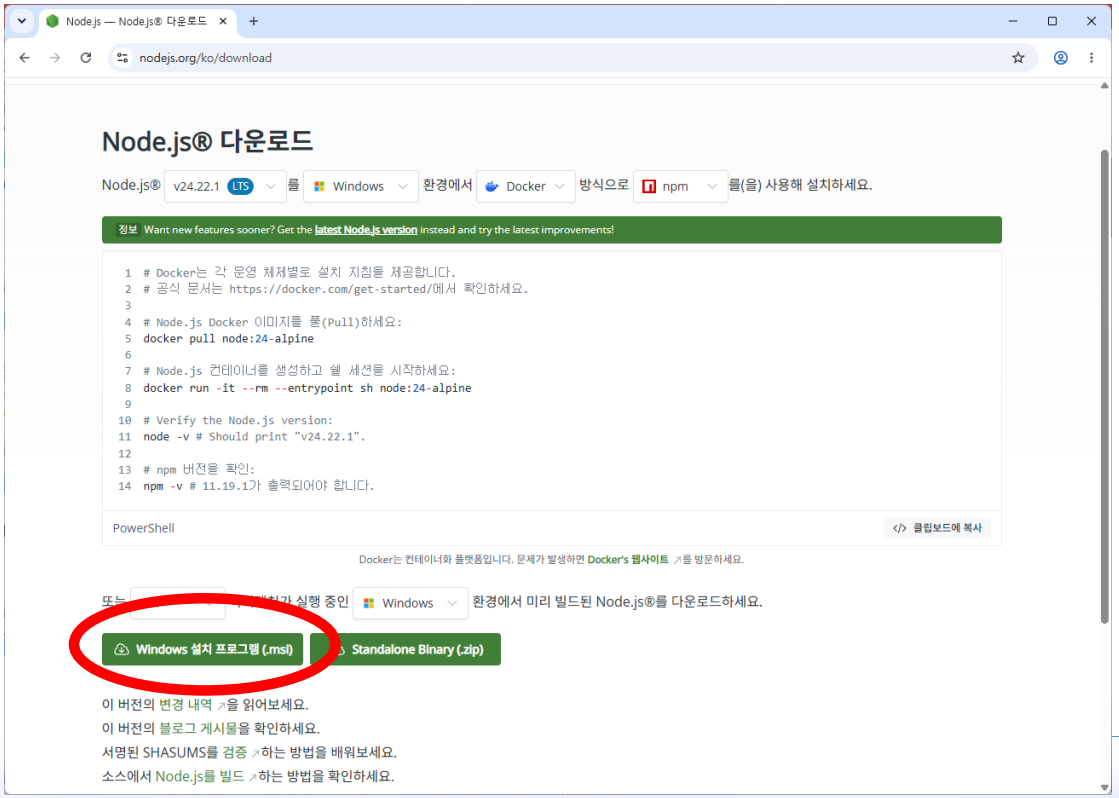
```
PS C:\> winget install OpenJS.NodeJS.LTS     # 처음 설치
PS C:\> winget upgrade OpenJS.NodeJS.LTS     # 업데이트
```

※ 버전 숫자는 예시, 관리자 권한 창이 뜨면 "예", 설치 후 VS Code를 닫았다가 다시 열고 node -v 로 확인

※ winget이 없으면 https://nodejs.org 에서 LTS 설치 파일(.msi)을 받아 설치 (기존 버전 위에 설치하면 업데이트)

개발 환경

- **설치 준비 Node.js**
  - ✓ **winget이 없거나 설치가 어려운 경우,**
  - ✓ **https://nodejs.org/ko/download 에서 window 설치 프로그램 다운로드 받아서 설치**



개발 환경

Starter 프로젝트: 실습 폴더

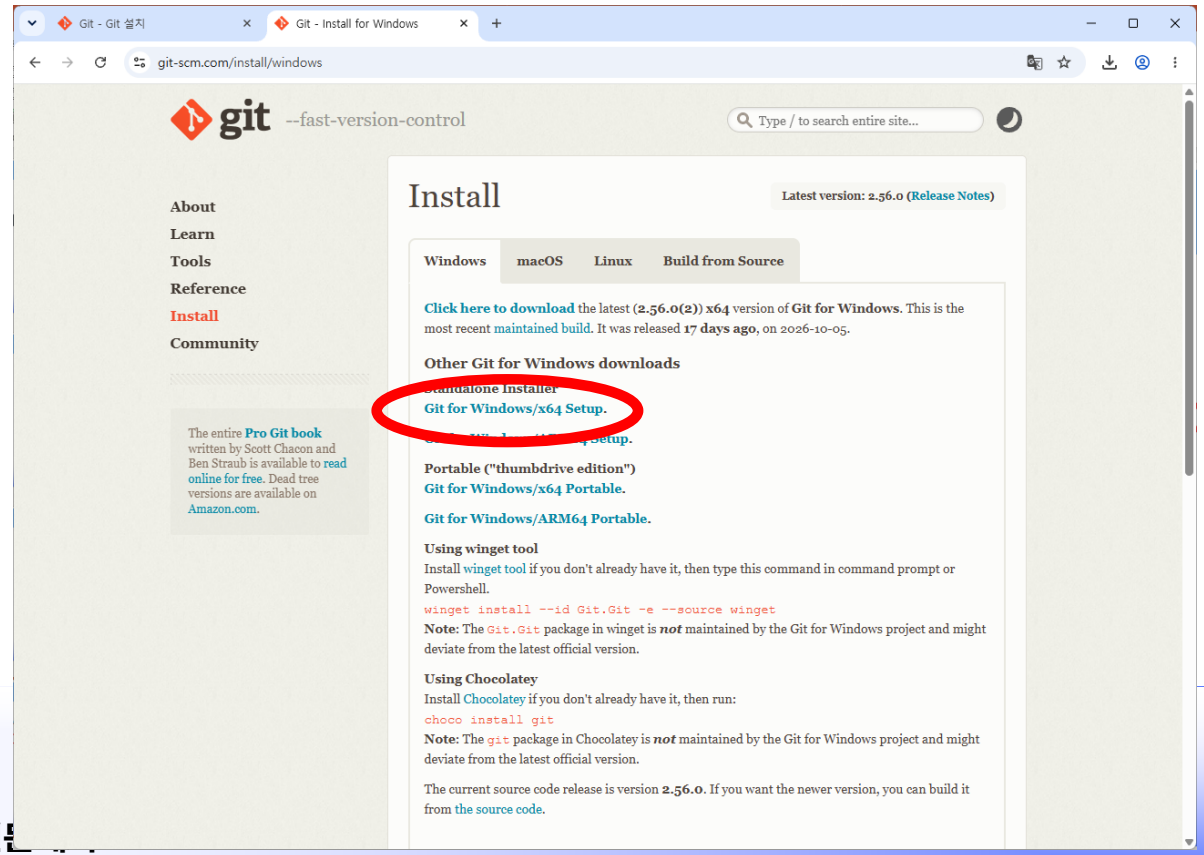
```
hardhat-test/
├─ package.json          ← 설치할 패키지 목록
├─ package-lock.json     ← 모두 같은 버전 설치
├─ hardhat.config.ts     ← Hardhat 설정
├─ tsconfig.json         ← TypeScript 설정
├─ README.md             ← 명령 요약
├─ contracts/            ← .sol 파일 (Counter.sol …)
├─ ignition/modules/     ← 배포 모듈 (Counter.ts …)
├─ scripts/              ← ethers.js 호출 (interact.js …)
└─ test/                 ← (부록) 테스트
```

Starter 받기 (GitHub) – 실습을 위해 따로 만든 간단한 source

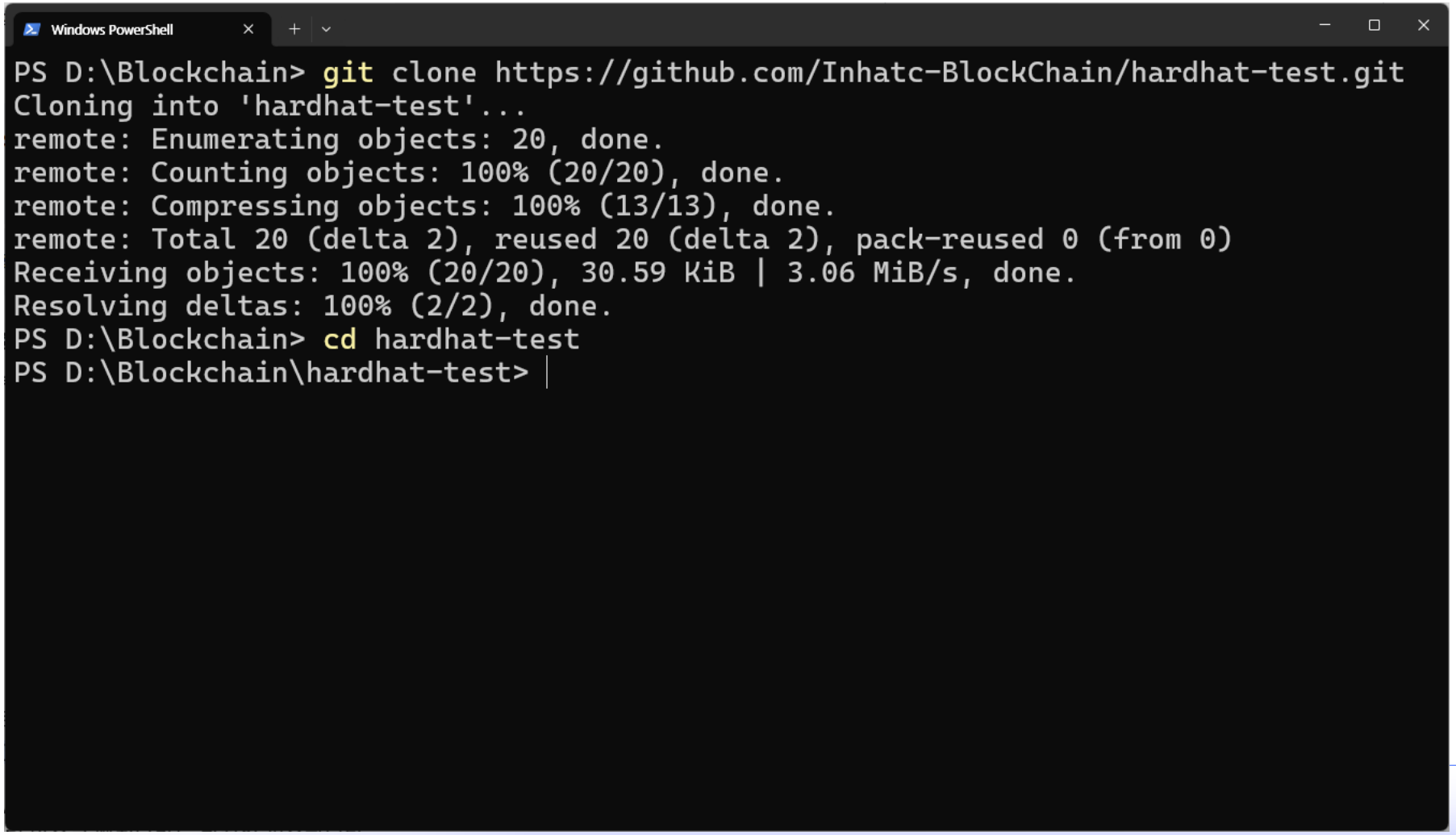
- ✓ C: 나 D: Blockchain 폴더(디렉토리) 만들기
- ✓ github.com/Inhatc-BlockChain/hardhat-test
- ✓ git clone https://github.com/Inhatc-BlockChain/hardhat-test.git
- ✓ 또는 , Code → Download ZIP → 압축 풀기

개발 환경

- **Starter 받기 (GitHub)**
  - ✓ **git이 없거나 설치가 어려운 경우,**
  - ✓ https://git-scm.com/download/win **에서**
    window 설치 프로그램 (Git for Windows/x64 Setup.**) 다운로드 받아서 설치**

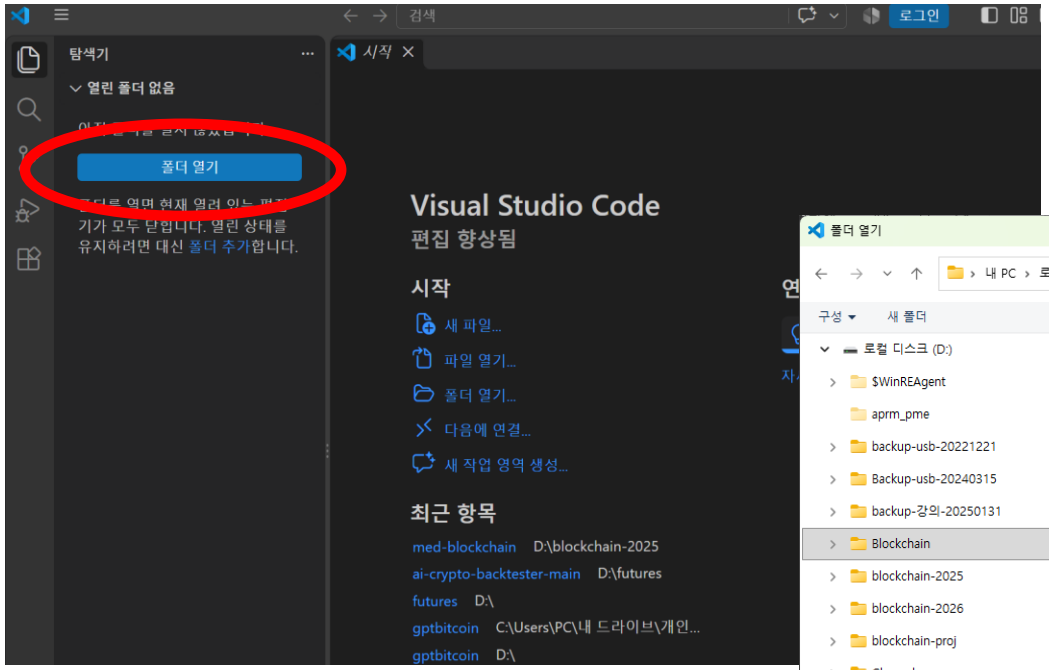


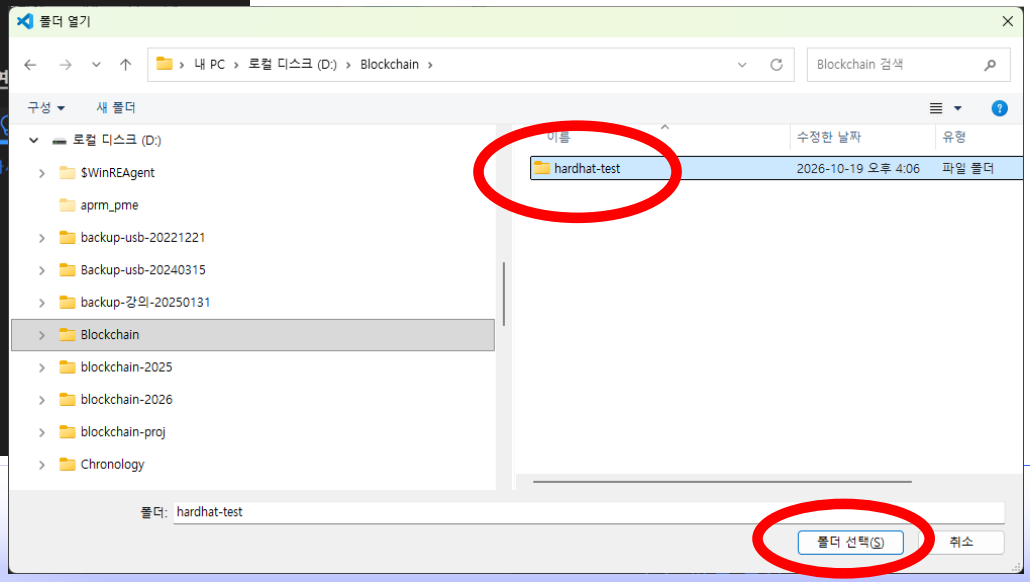
개발 환경


Windows PowerShell
PS D:\Blockchain> git clone https://github.com/Inhatc-BlockChain/hardhat-test.git
Cloning into 'hardhat-test'...
remote: Enumerating objects: 20, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (13/13), done.
remote: Total 20 (delta 2), reused 20 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (20/20), 30.59 KiB | 3.06 MiB/s, done.
Resolving deltas: 100% (2/2), done.
PS D:\Blockchain> cd hardhat-test
PS D:\Blockchain\hardhat-test>

개발 환경

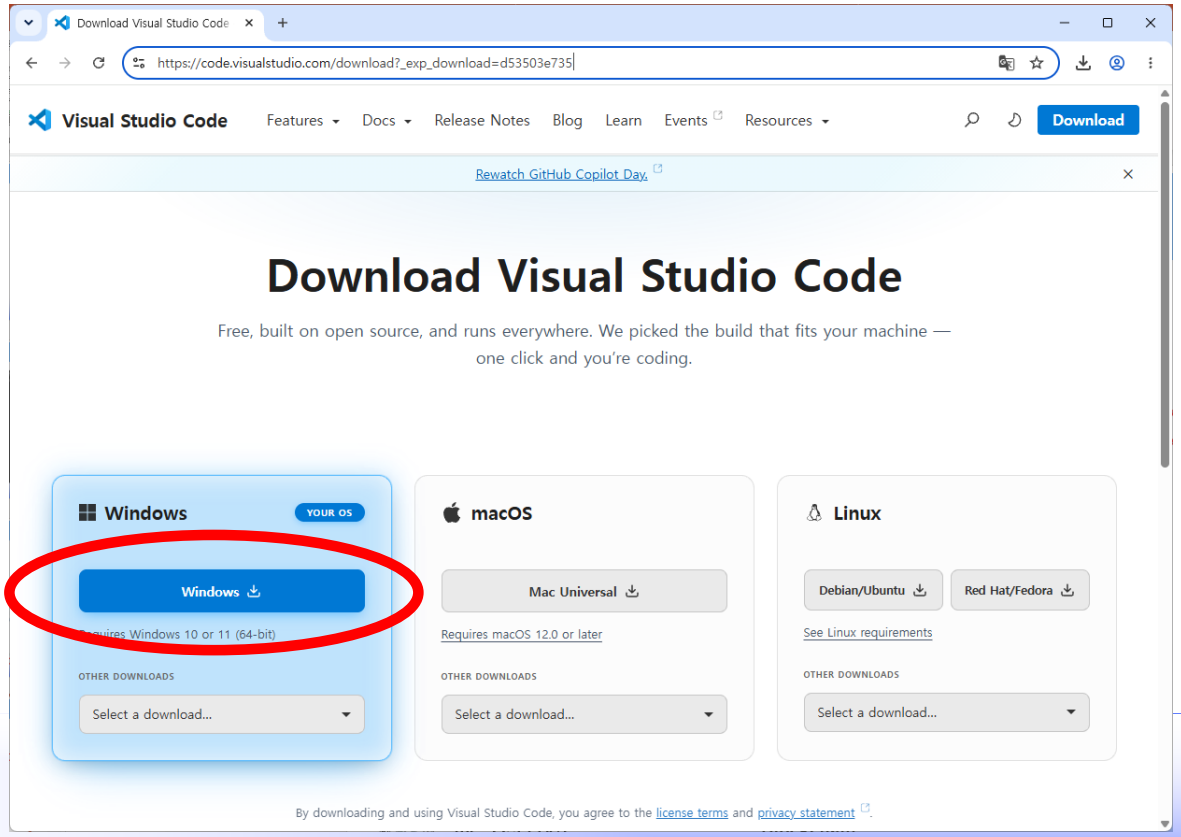
- **VS Code 사용**
  - ✓ hardhat-test 폴더 열기
    - ✓ (OneDrive, Google Drive 동기화 폴더 안에서는 실습하지 않는 것이 좋음
  - ✓ 폴더 열기: File → Open Folder → hardhat-test
  - ✓ 터미널 열기: Terminal → New Terminal



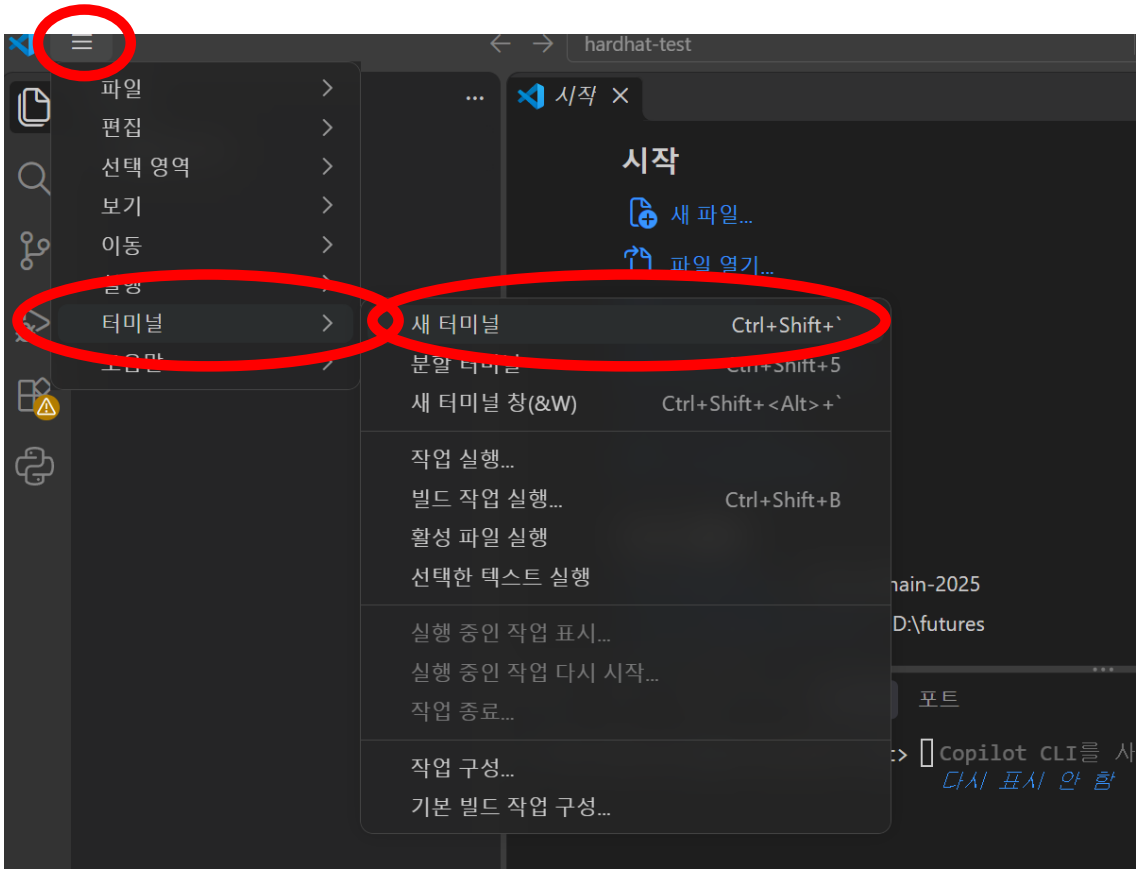


개발 환경

- **VS Code 사용**
  - ✓ **Vs code가 없는 경우,**
  - ✓ https://code.visualstudio.com/download?_exp_download=d53503e735 **에서 window 설치 프로그램 다운로드 받아서 설치**



개발 환경


hardhat-test
파일
편집
선택 영역
보기
이동
터미널
새 터미널 Ctrl+Shift+`
새 터미널 창(&W) Ctrl+Shift+<Alt>+`
작업 실행...
빌드 작업 실행... Ctrl+Shift+B
활성 파일 실행
선택한 텍스트 실행
실행 중인 작업 표시...
실행 중인 작업 다시 시작...
작업 종료...
작업 구성...
기본 빌드 작업 구성...
시작
새 파일...
파일 열기...
D:\futures
포트
Copilot CLI를
다시 표시 안 함

개발 환경

- **설치 준비 PowerShell 실행 정책 – 문제가 생길 경우,**
  - ✓ **Windows 기본 설정에서는 PowerShell이 npm, npx 실행을 막을 수 있음**

```
PS C:\blockchain\hardhat-test> npx hardhat build
npx : 이 시스템에서 스크립트를 실행할 수 없으므로
C:\Program Files\nodejs\npx.ps1 파일을 로드할 수 없습니다.
```

- **해결 방법 (둘 중 하나)**
  - ✓ **① 내 계정에서만 실행 허용 (한 번만 실행)**

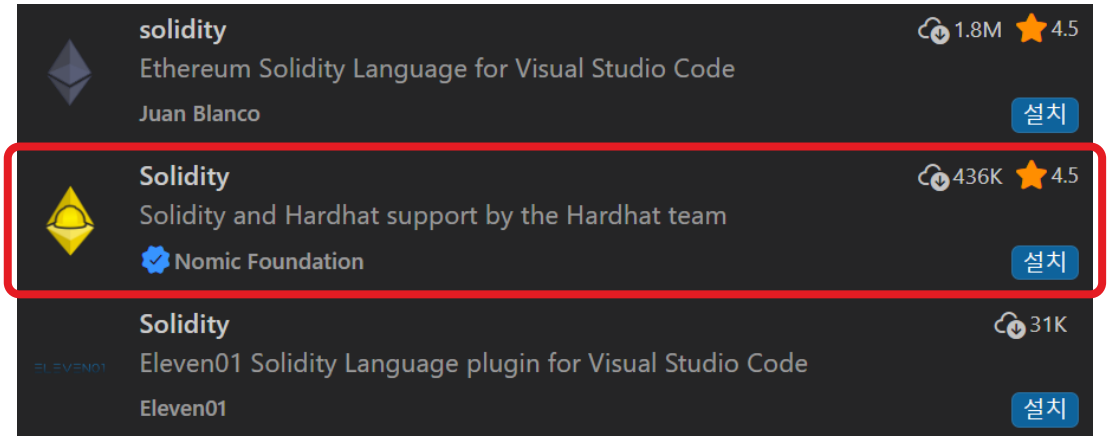
    Set-ExecutionPolicy -Scope CurrentUser RemoteSigned

```
PS C:\> Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

  - ✓ **② VS Code 터미널 종류를 PowerShell 대신 Command Prompt(cmd)로 변경**
    - ➢ **터미널 오른쪽 ∨ 버튼 → Command Prompt 선택**

개발 환경

- **설치 준비 – VS Code 확장: Solidity (Nomic Foundation)**
  - ✓ 왼쪽 확장 아이콘 (Ctrl + Shift + X) → "solidity" 검색
  - ✓ 게시자가 Nomic Foundation ✔ 인 것 (Hardhat 제작사) → 설치



← 이것 설치

  - ✓ 주의: 다른 Solidity 확장(예: Juan Blanco의 solidity)과 함께 설치하지 않기 → 하나만 사용

개발 환경

❑ 패키지 설치: npm ci

- ✓ 프로젝트 폴더로 이동한 뒤 실행

```
PS C:\blockchain> cd hardhat-test
PS C:\blockchain\hardhat-test> npm ci
added 187 packages, and audited 188 packages in 9s
```

※ 시간은 PC, 네트워크마다 다름. npm warn 으로 시작하는 줄은 경고(실패 아님)

명령	동작	사용
`npm ci`	package-lock.json에 적힌 버전 그대로 설치	수업 (모두 같은 환경)
`npm install`	package.json을 보고 설치, lock 파일 갱신	새 패키지 추가할 때

- ✓ 설치 후 `node_modules/` 폴더 생성 (약 200MB, 실제 패키지가 들어 있음)
- ✓ `node_modules/`는 다른 사람에게 전달하지 않음 → 받은 사람이 다시 `npm ci`

개발 환경

- package.json: 프로젝트 정보와 사용할 패키지 목록

```
{
  "name": "hardhat-test",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "build": "hardhat build",
    "test": "hardhat test",
    "node": "hardhat node",
    "deploy:local": "hardhat ignition deploy ignition/modules/Counter.ts --network localhost",
    "deploy:sepolia": "hardhat ignition deploy ignition/modules/Counter.ts --network sepolia",
    "interact": "node scripts/interact.js"
  },
  "devDependencies": {
    "@nomicfoundation/hardhat-toolbox-mocha-ethers": "4.0.0",
    "hardhat": "3.20.0"
  },
  "dependencies": {
    "ethers": "6.17.0"
  }
}
```

개발 환경

❑ package.json 항목 설명

항목	의미
`"type": "module"`	import 문법 사용 (최신 JavaScript 방식)
`devDependencies`	개발할 때만 필요한 도구 (Hardhat, Toolbox)
`dependencies`	프로그램 실행에 필요한 라이브러리 (ethers.js)
`"6.17.0"`	정확한 버전 고정 → 모두 같은 환경
`scripts`	긴 명령을 짧은 이름으로 등록

❑ scripts 사용: npm run 이름

짧은 명령	실제로 실행되는 명령
`npm run build`	npx hardhat build
`npm test`	npx hardhat test
`npm run deploy:local`	npx hardhat ignition deploy ignition/modules/Counter.ts --network localhost

개발 환경

❑ hardhat.config.ts: Hardhat 설정 파일 (Starter에 포함)

```
import { configVariable, defineConfig } from "hardhat/config";
import hardhatToolboxMochaEthers
  from "@nomicfoundation/hardhat-toolbox-mocha-ethers";

export default defineConfig({
  plugins: [
    hardhatToolboxMochaEthers    // ① 사용할 플러그인
  ],
  solidity: {
    version: "0.8.28"            // ② 컴파일러 버전
  },
  networks: {                    // ③ 접속할 네트워크
    localhost: {
      type: "http",
      chainType: "l1",
      url: "http://127.0.0.1:8545"
    },
    sepolia: {
      type: "http",
      chainType: "l1",
      url: configVariable("SEPOLIA_RPC_URL"),
      accounts: [
        configVariable("SEPOLIA_PRIVATE_KEY")
      ]
    }
  }
});
```

개발 환경

- **hardhat.config.ts에서 먼저 알아야 할 3가지**
  - ✓ ① `plugins`: 사용할 Hardhat 확장 기능 (Toolbox)
  - ✓ ② `solidity`: 사용할 Solidity 컴파일러 버전 (0.8.28)
  - ✓ ③ `networks`: 접속할 Ethereum 네트워크
- **networks 설명**
  - ✓ `localhost`: 내 PC의 Hardhat Node (주소 127.0.0.1, 포트 8545)
  - ✓ `sepolia`: 공개 테스트넷 → RPC URL과 개인 키가 필요
  - ✓ `configVariable("…")`: 비밀값을 코드에 직접 쓰지 않고 keystore에서 가져옴
    - ➢ 개인 키를 코드에 쓰면 파일 공유 시 유출 위험

Counter.sol

❑ contracts/Counter.sol 작성

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

contract Counter {

    uint256 public count;

    event Increment(uint256 newValue);

    function increment() public {
        count++;
        emit Increment(count);
    }

    function add(uint256 value) public {
        count += value;
        emit Increment(count);
    }
}
```

✓ 저장하는 습관 필요!

Counter.sol

❑ 코드 설명

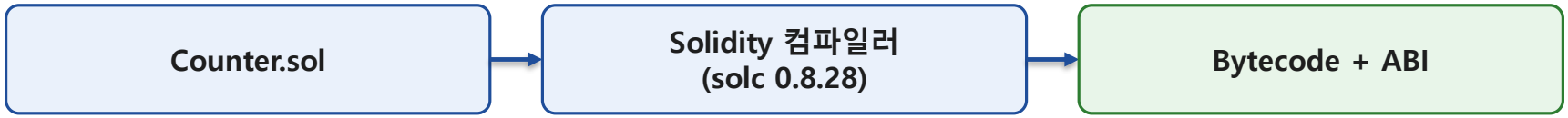
코드	의미
`uint256 public count;`	블록체인(storage)에 저장되는 상태 변수, public → 자동 getter count()
`event Increment(uint256 newValue);`	실행 사실을 로그로 기록하는 이벤트 선언
`increment()`	count를 1 증가 → 트랜잭션
`add(uint256 value)`	전달받은 숫자만큼 증가 → 트랜잭션
`emit Increment(count);`	변경된 값을 이벤트로 기록

Build

Build - npx hardhat build

```
PS C:\blockchain\hardhat-test> npx hardhat build
Downloading solc 0.8.28
Downloading solc 0.8.28 (WASM build)

Compiled 1 Solidity file with solc 0.8.28 (evm target: cancun)
```



- ✓ 처음 build할 때 컴파일러를 내려받음

Build

❑ Build 결과: artifacts 폴더

```
artifacts/
└─ contracts/
   └─ Counter.sol/
      └─ Counter.json    ← ABI + Bytecode
```

- ✓ cache/, artifacts/는 build 때마다 다시 만들어짐


```
"abi": [
  {
    "inputs": [],
    "name": "count",
    "outputs": [ { "type": "uint256", … } ],
    "stateMutability": "view",
    "type": "function"
  },
  {
    "inputs": [],
    "name": "increment",
    "outputs": [],
    "stateMutability": "nonpayable",
    "type": "function"
  }, …
],
"bytecode": "0x608060405234801560e575f5ffd5b50…"
```


- ✓ Bytecode: EVM이 실행하는 프로그램 (배포되는 코드)
- ✓ ABI: 외부 프로그램이 컨트랙트의 함수 구조를 알기 위한 정보 (함수 이름, 입력, 출력)
- ✓ ethers.js가 이 ABI로 함수 호출 데이터(calldata)를 만듦

Hardhat Node

- 내 PC 안의 테스트용 Ethereum 네트워크 (≈ Remix VM)
  - npx hardhat node

```
PS C:\blockchain\hardhat-test> npx hardhat node
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
========

WARNING: Funds sent on live network to accounts with publicly known private keys WILL BE LOST.

Account #0:  0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1:  0x70997970c51812dc3a010c7d01b50e0d17dc79c8 (10000 ETH)
...
```



- ✓ 테스트 계정 20개, 각 10,000 ETH (가짜 ETH)
- ✓ 주의: 이 개인 키들은 전 세계에 공개된 테스트용 → 실제 지갑에서 절대 사용 금지

Hardhat Node

❑ 터미널 2개 사용 – npx hardhat node

터미널 1
npx hardhat node
(끄지 않고 계속 실행)

터미널 2
배포, ethers.js 실행 등
나머지 명령

- ✓ VS Code 터미널 오른쪽 + 버튼 → 두 번째 터미널
- ✓ Hardhat Node를 정지하면 블록체인 내용이 모두 사라짐

Ignition

ignition/modules/Counter.ts 작성: 무엇을 배포할지 선언

```
import { buildModule }
  from "@nomicfoundation/hardhat-ignition/modules";

export default buildModule(
  "CounterModule",              // 모듈 이름
  (m) => {

    const counter =
      m.contract("Counter");    // Counter 컨트랙트를 배포하겠다고 선언

    return { counter };         // 배포 결과를 돌려줌
  }
);
```

- ✓ `m.contract("Counter")`: artifacts의 Counter를 배포하겠다는 "계획"
- ✓ Ignition은 단순히 한 번 보내는 것이 아니라 **배포 기록을 저장하고 관리**
  - ➢ 같은 배포를 반복하지 않음, 실패하면 이어서 진행

Ignition

❑ m은 어디서 오고, 언제 쓰일까?

개발자가 작성한 코드

```
buildModule(
  "CounterModule",
  (m) => {
    const counter =
      m.contract("Counter");
    return { counter };
  }
);
```

함수를 전달 →

① buildModule 이 배포 모듈을 구성

↓

② Ignition 이 도구 객체 m 을 준비

↓

③ (m) => { ... } 함수를 m 과 함께 호출

④ m.contract("Counter") → 배포 대상 정의 → return { counter } → 배포 대상을 외부에 공개

Ignition

❑ 참고: m(모듈 빌더)으로 할 수 있는 일

메서드	하는 일
`m.contract("이름", [인자])`	컨트랙트를 새로 배포
`m.contractAt("이름", 주소)`	이미 배포된 컨트랙트를 주소로 연결 (배포 X)
`m.library("이름")`	라이브러리를 배포
`m.call(컨트랙트, "함수명", [인자])`	컨트랙트의 함수를 호출 (트랜잭션 발생)
`m.staticCall(컨트랙트, "함수명", [인자])`	view / pure 함수로 값을 읽음 (트랜잭션 없음)
`m.send("id", 주소, 금액)`	ETH 또는 데이터를 계정으로 전송
`m.getAccount(번호)`	Hardhat Node의 테스트 계정 (0번, 1번, ...)

- ✓ **모두 바로 실행되는 것이 아니라 "이런 작업을 하겠다" 는 계획을 등록**
  - ➢ **npx hardhat ignition deploy 할 때 필요한 순서대로 실행되고 배포 기록에 저장**

Ignition

❑ 로컬 배포 (터미널 2)

npx hardhat ignition deploy ignition/modules/Counter.ts --network localhost

```
PS C:\blockchain\hardhat-test> npx hardhat ignition deploy ignition/modules/Counter.ts --network localhost

Hardhat Ignition 🚀

Deploying [ CounterModule ]

Batch #1
  Executed CounterModule#Counter

[ CounterModule ] successfully deployed 🚀

Deployed Addresses

CounterModule#Counter - 0x5FbDB2315678afecb367f032d93F642f64180aa3
```

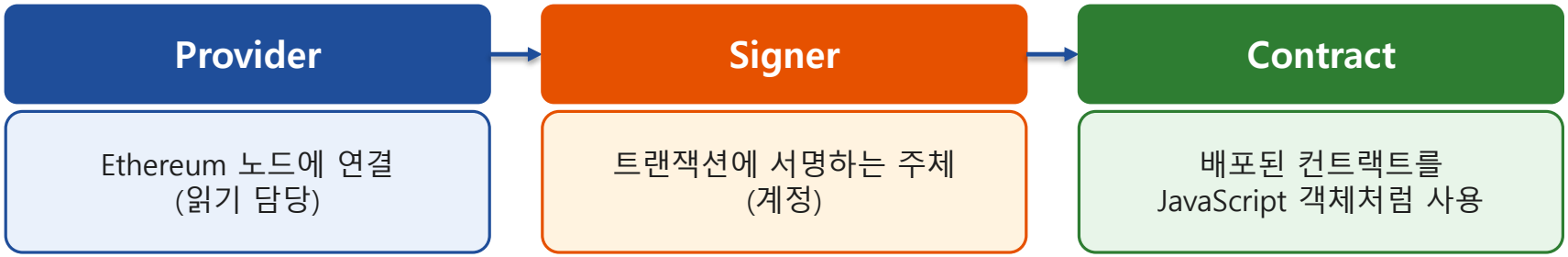
ignition/deployments/chain-31337/deployed_addresses.json

```
{
  "CounterModule#Counter": "0x5FbDB2315678afecb367f032d93F642f64180aa3"
}
```

- ✓ chain-31337: Hardhat Node의 chain ID, 배포 주소가 파일에 기록됨
- ✓ Node를 새로 시작하면 첫 배포 주소는 항상 0x5FbDB… (같은 계정의 첫 트랜잭션이므로)

ethers.js

- **ethers.js 핵심 3가지: Provider → Signer → Contract**



- ✓ **비유: Provider = 은행 지점 연결, Signer = 내 도장(서명), Contract = 사용할 창구**
- ✓ **Contract를 만들 때 필요한 것: 주소(address) + ABI + Provider 또는 Signer**



scripts/interact.js 작성 ① 연결 준비

```
import fs from "node:fs";
import { ethers } from "ethers";

// 1. Provider: 내 PC의 Hardhat Node 에 연결
const provider =
  new ethers.JsonRpcProvider("http://127.0.0.1:8545");

// 2. Signer: Hardhat Node 가 관리하는 0번 테스트 계정
const signer =
  await provider.getSigner(0);

// 3. 배포 주소: Ignition 배포 기록 파일에서 자동으로 읽기
const deployed = JSON.parse(
  fs.readFileSync(
    "ignition/deployments/chain-31337/deployed_addresses.json", "utf8"));
const address = deployed["CounterModule#Counter"];

// 4. ABI: 사용할 함수 목록 (사람이 읽을 수 있는 형태)
const abi = [
  "function count() view returns (uint256)",
  "function increment()",
  "function add(uint256)"
];

// 5. Contract: 배포된 컨트랙트를 JavaScript 객체처럼 사용
const counter = new ethers.Contract(address, abi, signer);
```

❑ scripts/interact.js 작성 ② 함수 호출
- node scripts/interact.js

```
console.log("Contract 주소:", address);
console.log("서명 계정:", await signer.getAddress());

// view 함수 → call (트랜잭션 X, 가스비 X)
console.log("현재 값:", await counter.count());

// 상태 변경 함수 → 트랜잭션 (서명 O, 가스 사용)
const tx = await counter.increment();  // increment()를 실행하는 트랜잭션 자체를 받음 - 아직 블록에 반영전
const receipt = await tx.wait();       // 트랜잭션이 블록에 포함될 때까지 기다림,  receipt.logs : event 로그

console.log("트랜잭션 해시:", tx.hash);
console.log("사용한 가스:", receipt.gasUsed);
console.log("변경 후:", await counter.count());
```

```
PS C:\blockchain\hardhat-test> node scripts/interact.js
Contract 주소: 0x5FbDB2315678afecb367f032d93F642f64180aa3
서명 계정: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
현재 값: 0n
트랜잭션 해시: 0x226a6a0f54a4126ec0306b3e77c971d0af0cf2ea49891d6361bf78c5c6c385d8
사용한 가스: 44584n
변경 후: 1n
```

※ 0n, 1n 의 n = BigInt (큰 정수). uint256은 일반 숫자로 표현할 수 없을 만큼 커서 ethers.js는 BigInt로 돌려줌

ethers.js

- 0n ?
  - ✓ JavaScript의 BigInt (큰 정수) 표기: `0n`, `1n`
  - ✓ uint256은 일반 숫자(Number)로 표현할 수 없을 만큼 큼 → ethers.js는 BigInt 사용

❑ receipt: 트랜잭션 실행이 블록에 담긴 결과

항목	의미	interact.js 실행 예
receipt.status	성공 여부 (1이면 성공, 0이면 실패)	1
receipt.blockNumber	트랜잭션이 담긴 블록 번호	2 (1번 블록 = 배포)
receipt.gasUsed	실제로 사용한 가스량	44584n
receipt.fee	가스비 = gasUsed × gasPrice (wei)	78742192726576n
receipt.from, to	보낸 계정, 받은 주소 (컨트랙트)	0xf39F..., 0x5FbDB...
receipt.contractAddress	배포 트랜잭션이면 새 컨트랙트 주소	null (함수 호출)
receipt.logs	실행 중 emit된 이벤트 로그	1개 (Increment)
receipt.hash	트랜잭션 해시 (= tx.hash)	0x226a6a0f...

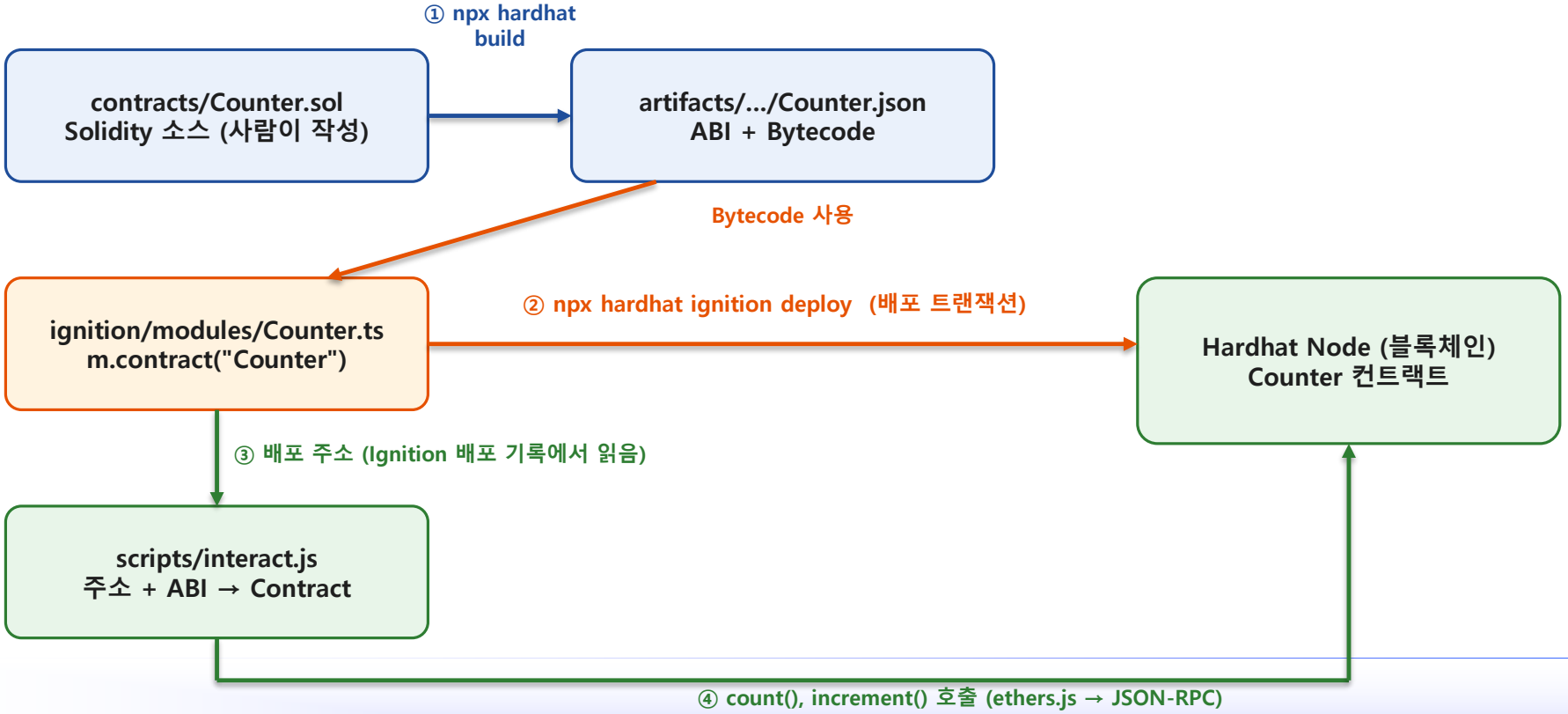
```
// ABI에 event를 추가하면 logs를 이벤트 이름과 값으로 볼 수 있음
const abi = [ /* … */, "event Increment(uint256 newValue)" ];
console.log(receipt.logs[0].eventName, receipt.logs[0].args);   // Increment Result(1) [ 1n ]
```

※ 로컬 Hardhat Node에서 interact.js(V1)를 처음 실행한 결과. ABI에 event가 없으면 logs는 해석 전 원본(topics, data)

ethers.js

- **파일 관계 :**
  Counter.sol은 컨트랙트 코드, interact.js는 컨트랙트의 함수 호출

Counter.json : 컴파일 결과 (ABI, Bytecode), Counter.ts : 배포 방법



ethers.js

call과 트랜잭션의 차이

	counter.count()	counter.increment()
함수 종류	view	상태 변경
보내는 방식	call (eth_call)	트랜잭션 (eth_sendTransaction)
서명	필요 없음	Signer가 서명
가스비	없음	있음
결과	값을 바로 받음 (0n)	트랜잭션 객체 → tx.wait()로 완료 대기

interact.js를 두 번 실행해 보기

- ✓ 1번째: 사용한 가스 44,584 (count를 0 → 1, 비어 있던 storage에 처음 저장)
- ✓ 2번째: 사용한 가스 27,484 (1 → 2, 이미 값이 있는 storage 변경)
- ✓ → Gas : storage에 처음 값을 저장할 때 가스가 더 많이 듦

ethers.js

❑ Signer는 상황에 따라 만드는 방법이 다름

로컬 (Hardhat Node가 계정을 관리)

```
const signer = await provider.getSigner(0);
```

Sepolia (내가 개인 키로 직접 서명)

```
const signer = new ethers.Wallet(PRIVATE_KEY, provider);
```

웹 브라우저 + MetaMask (MetaMask가 서명)

```
const provider = new ethers.BrowserProvider(window.ethereum);
const signer = await provider.getSigner();
```

✓ Signer ≠ 개인 키 그 자체, Signer = 트랜잭션에 서명할 수 있는 주체

Sepolia

- **Sepolia 배포 준비물**
  - ✓ ① Sepolia ETH가 있는 MetaMask 실습 계정
  - ✓ ② RPC URL: Sepolia 노드에 접속하는 주소
    - 공개 RPC (가입 불필요): `https://ethereum-sepolia-rpc.publicnode.com`
    - 또는 Infura, Alchemy 가입 후 발급 (더 안정적)
  - ✓ ③ 실습 계정의 개인 키 (MetaMask에서 내보내기)

Sepolia

- **keystore: 비밀값을 암호화해서 저장**
  npx hardhat keystore set SEPOLIA_RPC_URL

12341234

```
PS D:\Blockchain\hardhat-test> npx hardhat keystore set SEPOLIA_RPC_URL

🧑‍🔒 Hardhat Production Keystore 🔐🧑

This is the first time you are using the production keystore, please set a password.
The password must have at least 8 characters.

[hardhat-keystore] Enter the password: ********
[hardhat-keystore] Please confirm your password: ********
[hardhat-keystore] Enter secret to store in the production keystore: ************************
**********
```

https://ethereum-sepolia-rpc.publicnode.com

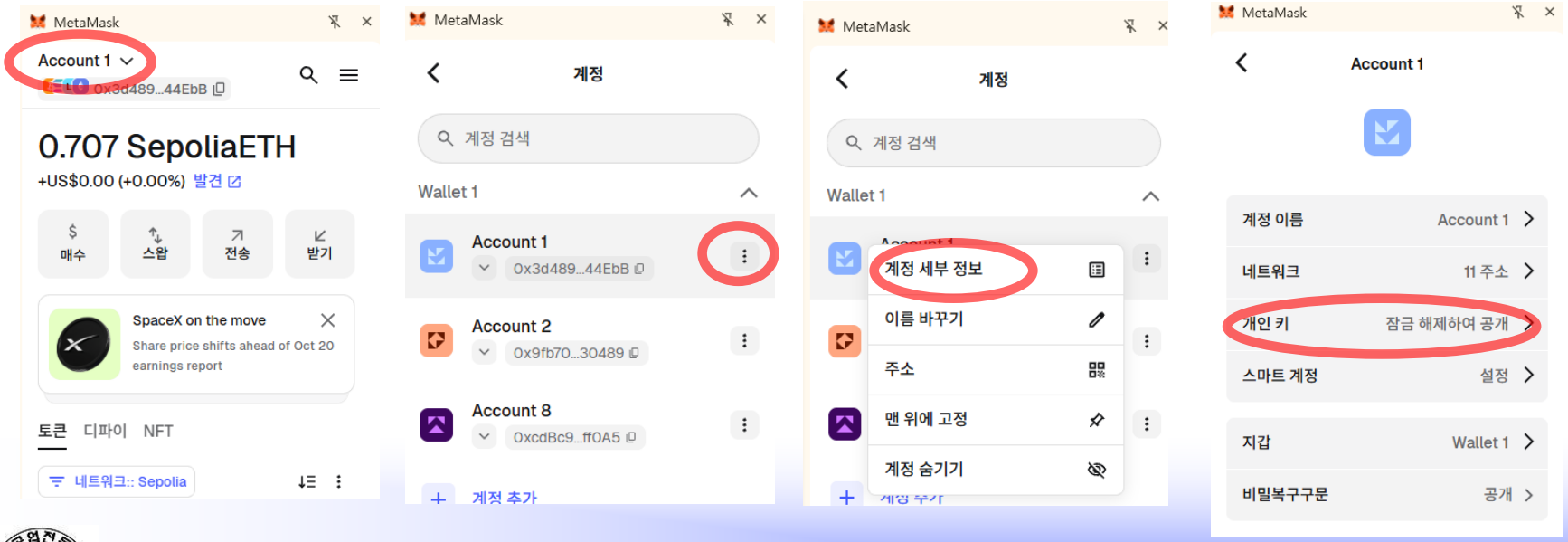
- ✓ 처음 한 번은 keystore 비밀번호 만들기 (8자 이상) → 이후 사용할 때마다 입력
- ✓ hardhat.config.ts의 `configVariable("SEPOLIA_RPC_URL")`이 이 값을 읽음
- ✓ 확인, 삭제: `npx hardhat keystore list`, `npx hardhat keystore delete SEPOLIA_PRIVATE_KEY`

Sepolia

❑ 개인키

MetaMask 실습 계정 → 계정 메뉴(⋮) → 계정 세부 정보 → 개인 키 표시 → 비밀번호 입력 → 복사

- ✓ **개인 키 = 계정의 모든 권한**, 다른 사람에게 보여 주거나 코드, 파일에 저장하지 않기
- ✓ 반드시 실제 자산이 없는 **실습 전용 계정**만 사용
- ✓ MetaMask의 개인 키는 0x 없이 표시됨 → 그대로 사용 가능
  - ➢ 메뉴 이름은 MetaMask 버전에 따라 조금 다를 수 있음



Sepolia

- **keystore: 비밀값을 암호화해서 저장**
  npx hardhat keystore set SEPOLIA_PRIVATE_KEY

```
PS D:\Blockchain\hardhat-test> npx hardhat keystore set SEPOLIA_PRIVATE_KEY
[hardhat-keystore] Enter the password: ********
[hardhat-keystore] Enter secret to store in the production keystore: **********************
**********************************************
Key "SEPOLIA_PRIVATE_KEY" set in the production keystore
PS D:\Blockchain\hardhat-test>
```

Metamask 계정의 개인키

- ✓ hardhat.config.ts의 `configVariable("SEPOLIA_PRIVATE_KEY")`이 이 값을 읽음
- ✓ 확인, 삭제: `npx hardhat keystore list`, `npx hardhat keystore delete SEPOLIA_PRIVATE_KEY`
- ✓ 저장 위치: `%APPDATA%\hardhat-nodejs\Config\keystore.json` (Windows 사용자별)
  - ➢ 실습실 공용 PC: 수업 후 delete로 정리

Sepolia

❑ Sepolia 배포

npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia

```
PS C:\blockchain\hardhat-test> npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia
Enter the password: ********
? Confirm deploy to network sepolia (11155111)? » (y/N)  y

Hardhat Ignition 🚀

Deploying [ CounterModule ]

Batch #1
  Executed CounterModule#Counter

[ CounterModule ] successfully deployed 🚀

Deployed Addresses

CounterModule#Counter - 0x…(각자 다른 주소)
```

※ 화면 예시 (입력 순서와 문구는 버전에 따라 조금 다를 수 있음)

- ✓ keystore 비밀번호 입력 → 배포 확인 질문에 y 입력
- ✓ Sepolia는 블록이 약 12초마다 생성 → 로컬보다 오래 걸림 (수십 초)
- ✓ 배포 기록: `ignition/deployments/chain-11155111/deployed_addresses.json`
- ✓ 중간에 실패해도 같은 명령을 다시 실행하면 이어서 진행

Sepolia

❑ Local과 Sepolia 비교



	Local (Hardhat Node)	Sepolia
chain ID	31337	11155111
ETH	가짜 10,000 ETH	Faucet에서 받은 테스트 ETH
서명 계정	Node가 제공하는 테스트 계정	내 MetaMask 실습 계정 (keystore)
결과 확인	터미널	https://sepolia.etherscan.io 에서 주소 검색

- ✓ **Etherscan에서 배포 트랜잭션, Gas 사용량 확인**

Sepolia

❑ 개인 키로 직접 서명: interact.js → interact-wallet.js

	interact.js (로컬)	interact-wallet.js (Sepolia)
① **Provider**	JsonRpcProvider("http://127.0.0.1:8545")	JsonRpcProvider("https://ethereum-sepolia-rpc.publicnode.com")
② **Signer**	await provider.getSigner(0)	new ethers.Wallet(process.env.PRIVATE_KEY, provider)
③ **배포 주소**	ignition/deployments/chain-31337/...	ignition/deployments/chain-11155111/...

```
PS C:\blockchain\hardhat-test> $env:PRIVATE_KEY = "실습 계정 개인 키"
PS C:\blockchain\hardhat-test> node scripts/interact-wallet.js
```

- ✓ **node scripts/interact-wallet.js 실행시키기 전에, $env:PRIVATE_KEY = "실습 계정 개인 키" 꼭 실행시켜야 함**
- ✓ **$env:PRIVATE_KEY: 이 터미널에서만 유효 (닫으면 사라짐), 코드, 파일에 개인 키를 쓰지 않기**
- ✓ **keystore는 npx hardhat 명령에서만 읽을 수 있음 → node로 실행하는 스크립트는 환경 변수 사용**
- ✓ **ABI, Contract, count(), increment() 부분은 interact.js와 그대로 같음**

Sepolia

❑ scripts/interact-wallet.js 작성 ① 연결 준비

```
// interact.js 를 Sepolia + 내 개인 키(ethers.Wallet) 용으로 바꾼 버전
//   PowerShell:  $env:PRIVATE_KEY = "개인 키"
//                node scripts/interact-wallet.js
import fs from "node:fs";
import { ethers } from "ethers";

// 1. Provider: 127.0.0.1:8545 대신 Sepolia RPC URL
const RPC_URL = process.env.RPC_URL ?? "https://ethereum-sepolia-rpc.publicnode.com";
const provider = new ethers.JsonRpcProvider(RPC_URL);

if (!process.env.PRIVATE_KEY) {
  console.log("PRIVATE_KEY 환경 변수가 없습니다.  PowerShell:  $env:PRIVATE_KEY = \"개인 키\"");
  process.exit(1);
}

// 2. Signer: provider.getSigner(0) 대신 내 개인 키
const signer = new ethers.Wallet(process.env.PRIVATE_KEY, provider);

// 3. 배포 주소: Ignition 배포 기록 (Sepolia = chain-11155111)
const { chainId } = await provider.getNetwork();
const address = JSON.parse(
  fs.readFileSync(`ignition/deployments/chain-${chainId}/deployed_addresses.json`, "utf8")
)["CounterModule#Counter"];

// 4. ABI, Contract: interact.js 와 같음
const abi = [
  "function count() view returns (uint256)",
  "function increment()",
  "function add(uint256)"
];
const counter = new ethers.Contract(address, abi, signer);
```

Sepolia

scripts/interact-wallet.js 작성 ② 함수 호출

```
console.log("Contract 주소:", address);
console.log("서명 계정:", signer.address);
console.log("현재 값:", await counter.count());

const tx = await counter.increment();
console.log("트랜잭션 해시:", tx.hash);
const receipt = await tx.wait();   // Sepolia: 약 12초 이상

console.log("사용한 가스:", receipt.gasUsed);
console.log("변경 후:", await counter.count());
```

```
PS C:\blockchain\hardhat-test> node scripts/interact-wallet.js
Contract 주소: 0x5FbDB2315678afecb367f032d93F642f64180aa3
서명 계정: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
현재 값: 0n
트랜잭션 해시: 0xc2e1aa5287c96942825cd30296a9a126daa9f7221f5e1f12df5a5f0a67daafcd
사용한 가스: 44606n
변경 후: 1n
```

※ 로컬 테스트 노드로 확인한 화면 예시. Sepolia에서는 내 실습 계정, 46쪽에서 배포한 내 Contract 주소, 각자 다른 해시 (약 12초 이상)

Contract 수정

❑ 마지막 미션: Counter에 subtract() 추가

```
    function subtract(uint256 value) public {
        count -= value;
    }
```

npx hardhat build

npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia

```
PS C:\blockchain\hardhat-test> npx hardhat build
Compiled 1 Solidity file with solc 0.8.28 (evm target: cancun)

PS C:\blockchain\hardhat-test> npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia
[hardhat-keystore] Enter the password: ********
√ Confirm deploy to network sepolia (11155111)? ... yes
[ CounterModule ] Nothing new to deploy based on previous execution stored in
.\ignition\deployments\chain-11155111
Deployed Addresses
CounterModule#Counter - 0x…(46쪽에서 배포한 주소 그대로)
```

✓ 같은 명령으로 다시 배포했는데 **"새로 배포할 것이 없다"** 고 함 → 왜 그럴까?

Contract 수정

- 이유 ① Ignition이 배포 기록을 기억 → 같은 배포 반복 X
- 이유 ② 배포된 컨트랙트는 바뀌지 않음



- ✓ 새 버전은 새 주소로 배포해야 함

npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia --reset

```
PS C:\blockchain\hardhat-test> npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia --reset
[hardhat-keystore] Enter the password: ********
√ Confirm deploy to network sepolia (11155111)? ... yes

[ CounterModule ] successfully deployed 🚀
Deployed Addresses
CounterModule#Counter - 0x…(새 주소, 각자 다름)
```

- ✓ `--reset`: 이 네트워크(chain-11155111)의 모든 배포 기록을 지움, V1 컨트랙트는 블록체인에 남아 있음
- ✓ interact-wallet.js, subtract.js는 기록 파일에서 주소를 읽으므로 수정 없이 V2 사용

Contract 수정

❑ 0에서 1을 빼면? (ethers.js로 직접 호출)

```
const abi = [
  "function count() view returns (uint256)",
  "function subtract(uint256)"              // V2에 추가된 함수
];
const counter = new ethers.Contract(address, abi, signer);

console.log("현재 값:", await counter.count());
try {
  const tx = await counter.subtract(1n);   // 0 - 1 → underflow
  await tx.wait();
} catch (e) {
  console.log("실패:", e.shortMessage);
}
console.log("실행 후:", await counter.count());
```

```
PS C:\blockchain\hardhat-test> node scripts/subtract.js
현재 값: 0n
실패: execution reverted: Panic due to OVERFLOW(17)
실행 후: 0n
```

- ✓ Solidity 0.8 이상: 범위를 벗어난 계산은 자동으로 revert (Panic 17 = 0x11)
- ✓ ethers.js가 보내기 전 가스 예상(eth_estimateGas) 단계에서 실패를 알아냄 → 트랜잭션 전송 X, 가스비 X
- ✓ subtract.js도 실행 전에 $env:PRIVATE_KEY 설정 (48쪽)
- ✓ 테스트 코드로 자동화하는 것이 좋음 (Mocha, Chai 활용 등)

컨트랙트 추가

- 같은 프로젝트(hardhat-test)에 새 컨트랙트 추가하기
  - ✓ 새 폴더, npm ci 필요 없음 → 파일만 추가 (node_modules 하나를 같이 사용)

```
hardhat-test/
├─ contracts/
│   ├─ Counter.sol
│   └─ Storage.sol              ← ① 컨트랙트 추가
├─ ignition/modules/
│   ├─ Counter.ts
│   └─ Storage.ts               ← ② 배포 모듈 추가
├─ scripts/
│   ├─ interact.js
│   └─ interact-storage.js      ← ③ 호출 프로그램 추가
└─ test/
    └─ (부록) Storage.ts        ← ④ 테스트 추가 (선택)
```

- ✓ 예제: Storage = Remix 기본 예제 1_Storage와 같은 기능
- ✓ build는 contracts 폴더의 모든 .sol을 컴파일
  - ➢ Compiled 2 Solidity files
- ✓ 배포, 호출은 컨트랙트마다 따로

컨트랙트 추가

- ① contracts/Storage.sol ② ignition/modules/Storage.ts

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

contract Storage {   // contract 이름: 다른 파일과 겹치면 안 됨
    uint256 number;
    event Stored(uint256 newNumber);

    function store(uint256 num) public {
        number = num;
        emit Stored(num);
    }
    function retrieve() public view returns (uint256) {
        return number;
    }
}
```

```
import { buildModule }
  from "@nomicfoundation/hardhat-ignition/modules";

export default buildModule("StorageModule", (m) => {   // 모듈 이름도 겹치면 안 됨
  const storage = m.contract("Storage");
  return { storage };
});
```

```
PS C:\blockchain\hardhat-test> npx hardhat build
Compiled 2 Solidity files with solc 0.8.28 (evm target: cancun)
```

컨트랙트 추가

❑ ③ 배포하고 호출하기 (Counter와 같은 Sepolia)

```
PS C:\blockchain\hardhat-test> npx hardhat ignition deploy ignition/modules/Storage.ts --network sepolia
[ StorageModule ] successfully deployed 🚀
Deployed Addresses
CounterModule#Counter - 0x…(52쪽의 V2 주소)
StorageModule#Storage - 0x…(각자 다른 주소)
```

```
const address = deployed["StorageModule#Storage"];   // 기록 파일의 Storage 주소
const abi = [
  "function store(uint256)",
  "function retrieve() view returns (uint256)"
];
const storage = new ethers.Contract(address, abi, signer);
await (await storage.store(42n)).wait();
console.log("변경 후:", await storage.retrieve());
```

```
PS C:\blockchain\hardhat-test> node scripts/interact-storage.js
Contract 주소: 0x…(위의 Storage 주소)
현재 값: 0n
변경 후: 42n
```

- ✓ interact-storage.js 의 나머지 부분은 스스로 완성하기
- ✓ 실행 전에 $env:PRIVATE_KEY 설정 (48쪽)

컨트랙트 추가

❑ 여러 컨트랙트를 한 프로젝트에서 쓸 때 주의할 점 (실제 실행 결과)

상황	결과	지킬 것
한 파일에 문법 오류	HHE910: Compilation failed → 다른 컨트랙트도 build, 테스트, 배포 불가	오류 파일을 먼저 고치기
같은 contract 이름이 두 파일에	HHE1001: There are multiple artifacts for contract "Counter"	contract 이름은 모두 다르게
모듈 이름 중복 (buildModule("CounterModule", ...))	경고만 나오고 배포 기록이 섞임	모듈 이름은 모두 다르게
Counter만 --reset	Storage 기록도 지워짐 (deployed_addresses.json에서 사라짐)	Storage도 다시 배포

- ✓ **다른 실습(예: ERC20)도 같은 방법으로 contracts에 추가**

오류 해결

❑ 자주 나는 오류와 해결

화면에 보이는 메시지	원인	해결
스크립트를 실행할 수 없으므로 … npx.ps1	PowerShell 실행 정책	Set-ExecutionPolicy
No contracts to compile	contracts에 .sol 없음 / 저장 안 함	파일 위치, 저장 확인
connect ECONNREFUSED 127.0.0.1:8545	Hardhat Node 실행 안 함	터미널 1에서 npx hardhat node
could not decode result data (BAD_DATA)	주소에 컨트랙트가 없음	배포 다시 하기, 주소 확인
HHE7: Configuration Variable "SEPOLIA_RPC_URL" not found	keystore 값 없음	npx hardhat keystore set …
HHE10407: … insufficient funds	Sepolia ETH 부족	Faucet에서 ETH 받기
Nothing new to deploy	이전 배포 기록 있음	--reset 추가
TypeError: Member "x" not found	--init 템플릿의 Counter.t.sol	Counter.t.sol 삭제
HHE1001: multiple artifacts for contract …	같은 contract 이름이 두 파일에	contract 이름 바꾸기
HHE910: Compilation failed	어느 한 .sol 파일의 문법 오류	--> 표시된 파일, 줄 수정

정리

❑ 명령 정리

단계	명령	짧은 명령
설치	npm ci	
Build	npx hardhat build	npm run build
Test (부록)	npx hardhat test (--gas-stats)	npm test
Local Node	npx hardhat node	npm run node
Local 배포	npx hardhat ignition deploy ignition/modules/Counter.ts --network localhost	npm run deploy:local
ethers.js 호출	node scripts/interact.js	npm run interact
keystore	npx hardhat keystore set SEPOLIA_RPC_URL / SEPOLIA_PRIVATE_KEY	
Sepolia 배포	npx hardhat ignition deploy ignition/modules/Counter.ts --network sepolia	npm run deploy:sepolia
Sepolia 호출	$env:PRIVATE_KEY 설정 후 node scripts/interact-wallet.js (subtract.js, interact-storage.js)	
재배포	… --network sepolia --reset	
컨트랙트 추가	npx hardhat ignition deploy ignition/modules/Storage.ts --network sepolia	

정리

- 주요 사항
  - ✓ Hardhat: Build → Test → Deploy 를 명령으로 수행하는 개발 환경
  - ✓ Ignition: 배포 기록을 관리 → 같은 배포 반복 X, 새 버전은 새 주소
  - ✓ ethers.js 3가지: Provider(노드 연결) → Signer(서명) → Contract(주소 + ABI)
  - ✓ view 함수는 call (가스비 X), 상태 변경 함수는 트랜잭션 (tx.wait())
  - ✓ 개인 키는 코드에 쓰지 않고 keystore에 저장

- 다음 으로 하면 좋은 것
  - ✓ HTML + ethers.js + MetaMask: 웹 페이지에서 Counter 호출 (BrowserProvider)

정리

❑ 참고 자료 (클릭하면 이동)

자료	링크
실습 Starter source	github.com/Inhatc-BlockChain/hardhat-test
Hardhat 3 시작하기	hardhat.org/docs/getting-started
TypeScript + ethers.js 테스트	hardhat.org/docs/guides/testing/using-ethers
Hardhat Ignition 배포	hardhat.org/docs/guides/deployment/using-ignition
Hardhat 공식 플러그인	hardhat.org/docs/plugins/official-plugins
ethers.js v6 문서	docs.ethers.org/v6
Node.js 다운로드	nodejs.org
Sepolia Etherscan	sepolia.etherscan.io

참고 – chain ID

❑ chain ID 11155111은 어디서? → RPC 노드에 물어봄

```
// hardhat.config.ts
sepolia: {
  type: "http",
  chainType: "l1",
  url: configVariable("SEPOLIA_RPC_URL"),
  accounts: [configVariable("SEPOLIA_PRIVATE_KEY")]
}
```

sepolia는 이름일 뿐 (chainId 설정 없음)

① 비밀번호 입력 → keystore에서 SEPOLIA_RPC_URL 꺼냄

② 그 URL로 eth_chainId 요청 (JSON-RPC)

③ 노드 응답: 0xaa36a7 = 11155111

④ Confirm deploy to network sepolia (11155111)?

```
PS C:\blockchain\hardhat-test> $url  = "https://ethereum-sepolia-rpc.publicnode.com"
PS C:\blockchain\hardhat-test> $body = '{"jsonrpc":"2.0","method":"eth_chainId","params":[],"id":1}'
PS C:\blockchain\hardhat-test> Invoke-RestMethod -Method Post -Uri $url -ContentType "application/json" -Body $body

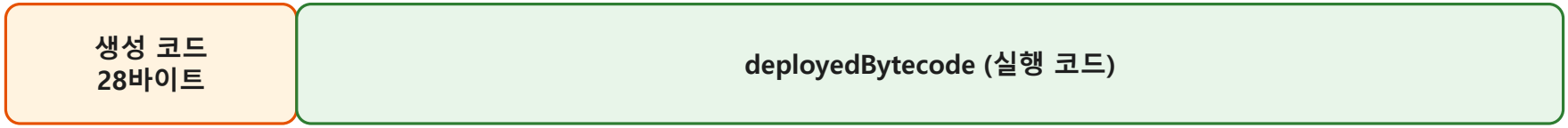
jsonrpc result   id
------- ------   --
2.0     0xaa36a7  1
```

- ✓ 로컬 Hardhat Node(127.0.0.1:8545)에 보내면 0x7a69 = 31337
- ✓ ethers.js도 같은 요청: (await provider.getNetwork()).chainId → chain-11155111 폴더
- ✓ config에 chainId: 11155111을 적으면 노드 응답과 비교 → 다르면 HHE708 오류

※ chain ID는 트랜잭션 서명에도 들어감 (EIP-155) → Sepolia용 트랜잭션을 다른 체인에서 재사용할 수 없음

참고 - Bytecode

❑ bytecode 안에 deployedBytecode가 들어 있음



bytecode 525바이트 = 배포 트랜잭션의 data (블록에 기록)

→ 생성 코드가 실행되어 뒤쪽 497바이트를 돌려줌 → 새 컨트랙트 주소의 code로 상태에 저장

```
0x6080604052            메모리 준비 (Solidity 기본 시작 코드)
  348015600e57 5f5ffd    ETH와 함께 배포하면 revert (생성자가 payable이 아님)
  5b50
  6101f1                실행 코드 길이 = 0x01f1 = 497바이트
  80 61001c             실행 코드 시작 위치 = 0x1c = 28번째 바이트
  5f 39                 CODECOPY: 자기 코드의 28번째부터 497바이트를 메모리로 복사
  5f f3                 RETURN: 그 497바이트를 돌려줌 → 상태에 실행 코드로 저장
  fe                    생성 코드의 끝 표시
```

- ✓ 생성자에 로직이 있으면 앞부분이 길어짐 (뒤에 실행 코드가 붙는 구조는 같음)
- ✓ 생성자에 인자가 있으면 그 값이 bytecode 맨 뒤에 붙어서 전송됨 (artifacts에는 없음)
- ✓ immutable이 있으면 배포할 때 실행 코드에 값을 채움 → 그 부분만 달라짐

※ artifacts/.../Counter.json, subtract 없는 Counter를 build한 결과 (코드가 바뀌면 바이트 수도 바뀜)

참고 – 네트워크 연결하기 스크립트

❑ 참고: Hardhat으로 실행하는 버전 (interact-network.ts)

```
import fs from "node:fs";
import hre from "hardhat";

const { ethers, networkName } = await hre.network.create();       // --network 로 선택한 네트워크
const { chainId } = await ethers.provider.getNetwork();

const deployed = JSON.parse(fs.readFileSync(
  `ignition/deployments/chain-${chainId}/deployed_addresses.json`, "utf8"));
const address = deployed["CounterModule#Counter"];

const [signer] = await ethers.getSigners();                       // 설정 파일의 계정
const counter = await ethers.getContractAt("Counter", address);   // ABI는 artifacts에서

console.log("네트워크:", networkName, "서명 계정:", signer.address);
console.log("현재 값:", await counter.count());
const receipt = await (await counter.increment()).wait();
console.log("사용한 가스:", receipt?.gasUsed, "변경 후:", await counter.count());
```

```
PS C:\blockchain\hardhat-test> npx hardhat run scripts/interact-network.ts --network localhost
```

- ✓ 같은 코드로 `--network sepolia`도 실행 가능 → 개인 키는 keystore에서 가져옴
- ✓ const deployed = {
  "CounterModule#Counter": "0x5FbDB2315678afecb367f032d93F642f64180aa3"
  };

참고 - Test

- 부록: Mocha + Chai 테스트 작성
  - ✓ 시간이 있을 때, 또는 과제, 복습용으로 사용
- 순서
  - ✓ test/Counter.ts 작성 → 코드 설명 → 테스트 실행
  - ✓ 테스트 더 작성해 보기 (add, 이벤트)
  - ✓ 가스 사용량 확인 (--gas-stats)
  - ✓ Underflow 테스트 (V2의 subtract)



참고 - Test

- **왜 테스트 코드를 작성할까?**
  - ✓ **Remix에서는 버튼을 하나씩 눌러 확인 → 코드를 고칠 때마다 다시 눌러야 함**
  - ✓ **테스트 코드는 명령 한 번으로 모든 기능을 자동 확인**
  - ✓ **배포 후에는 수정할 수 없는 스마트 컨트랙트 → 배포 전 테스트가 특히 중요**

코드	담당	역할
`describe(), it()`	Mocha	테스트를 묶고 하나씩 실행
`expect(…).to.equal(…)`	Chai	결과가 맞는지 검사
`ethers.deployContract()`	ethers.js	테스트용 컨트랙트 배포, 함수 호출
가상 블록체인	Hardhat	테스트할 때마다 새 블록체인을 만들어 실행

참고 - Test

❑ 테스트가 잡는 것: 에러 메시지가 나오지 않는 로직 오류

```
    function increment() public {
        count += 2;   // 실수! 원래는 count++ (1 증가)
        emit Increment(count);
    }
```

- ✓ 컴파일 성공
- ✓ 배포 성공
- ✓ 에러 메시지 없음

```
PS> npx hardhat build
Compiled 1 Solidity file with solc 0.8.28

PS> npx hardhat ignition deploy …
[ CounterModule ] successfully deployed 🚀
```

```
PS> npx hardhat test
  Counter
    ✔ 처음 count는 0이다
    1) increment()를 호출하면 1이 된다

  1 passing (27ms)
  1 failing

  1) Counter
       increment()를 호출하면 1이 된다:
     AssertionError: expected 2 to equal 1.
     + expected - actual
     -2
     +1
     at Context.<anonymous> (test\Counter.ts:29:11)
```

- ✓ 테스트만 실패
- ✓ expected 2 to equal 1
  - ➢ 실제 2, 기대 1
- ✓ + expected / - actual
  - ➢ + 기대값, - 실제값
- ✓ Counter.ts:29
  - ➢ 실패한 테스트 위치

참고 - Test

test/Counter.ts 작성

```
import { expect } from "chai";
import hre from "hardhat";

const { ethers } = await hre.network.create();   // Hardhat 블록체인 연결

describe("Counter", function () {

  it("처음 count는 0이다", async function () {
    const counter =
      await ethers.deployContract("Counter");    // 테스트용 배포
    expect(await counter.count())
      .to.equal(0n);                             // 0n = BigInt 0
  });

  it("increment()를 호출하면 1이 된다", async function () {
    const counter =
      await ethers.deployContract("Counter");
    const tx =
      await counter.increment();                 // 트랜잭션 보내기
    await tx.wait();                             // 블록에 담길 때까지 기다리기
    expect(await counter.count())
      .to.equal(1n);
  });

});
```

참고 - Test

- 테스트 코드 설명
  - ✓ `hre.network.create()`: Hardhat이 만든 테스트용 블록체인에 연결
    - ➢ 여기서 얻은 `ethers`는 Hardhat과 연결된 ethers.js
  - ✓ `describe("Counter", …)`: 관련 테스트 묶음, `it("…", …)`: 테스트 하나
  - ✓ `await`: 블록체인 작업은 시간이 걸림 → 결과가 올 때까지 기다림
  - ✓ `ethers.deployContract("Counter")`: artifacts를 이용해 테스트용으로 배포
    - ➢ 테스트마다 새로 배포 → 테스트끼리 서로 영향 없음
  - ✓ `tx.wait()`: 트랜잭션이 블록에 들어갈 때까지 기다림

참고 - Test

❑ 테스트 실행

```
PS C:\blockchain\hardhat-test> npx hardhat test
No contracts to compile

Running Solidity tests

Running Mocha tests

  Counter
    ✔ 처음 count는 0이다
    ✔ increment()를 호출하면 1이 된다

  2 passing (23ms)
```

- ✓ `No contracts to compile`: 이미 build 되어 있어 다시 컴파일할 것이 없다는 뜻
- ✓ `Running Solidity tests`: Solidity로 작성한 테스트(.t.sol)는 없으므로 비어 있음
- ✓ ✔ 표시 = 통과, 실패하면 빨간 글씨로 기대값(expected)과 실제값(actual) 표시
- ✓ 일부러 `.to.equal(2n)`으로 바꿔 실패 화면도 확인해 보기

참고 - Test

❑ 테스트 더 작성해 보기: add, 이벤트

```
it("add(5)를 호출하면 5가 된다", async function () {
  const counter =
    await ethers.deployContract("Counter");

  await (await counter.add(5n)).wait();

  expect(await counter.count())
    .to.equal(5n);
});

it("increment()는 Increment 이벤트를 발생시킨다", async function () {
  const counter =
    await ethers.deployContract("Counter");

  await expect(counter.increment())
    .to.emit(counter, "Increment")   // 이벤트 이름 검사
    .withArgs(1n);                   // 이벤트 값 검사
});
```

- ✓ `.to.emit(…).withArgs(…)`: Chai matcher (Toolbox에 포함) → 이벤트 발생과 값 검사

참고-Test

❑ 가스 사용량 확인: --gas-stats

```
PS C:\blockchain\hardhat-test> npx hardhat test --gas-stats
...
·····|·contracts/Counter.sol:Counter·|·····|·····|·····|·····|·····
·····|·Function name·|·Min·|·Average·|·Median·|·Max·|·#calls·|·····
·····|·add·|·45202·|·45202·|·45202·|·45202·|·1·|·····
·····|·count·|·23466·|·23466·|·23466·|·23466·|·3·|·····
·····|·increment·|·44791·|·44791·|·44791·|·44791·|·2·|·····
·····|·Deployment·|·Min·|·Average·|·Median·|·Max·|·#deployments·|·····
·····|·|·220920·|·220920·|·220920·|·220920·|·6·|·····
·····|·Bytecode size·|·777·|·|·|·|·|·····
```

- ✓ 함수별 가스 사용량을 표로 보여줌 (Gas 강의 내용과 연결)
- ✓ 배포(Deployment)가 가장 비쌈: 약 22만 gas (기본 21,000 + 배포 추가 32,000 + 코드 저장 …)
- ✓ count는 view 함수: 실제 사용 시에는 call이라 가스비 X (테스트에서는 비교용으로 측정)

Test – Contract 수정

❑ Underflow 테스트: 0에서 1을 빼면?

```
it("0에서 1을 빼면 revert된다", async function () {
  const counter =
    await ethers.deployContract("Counter");

  await expect(
    counter.subtract(1n)
  ).to.revert(ethers);                // 실패(revert)하는지 검사
});

it("revert 이유는 산술 오버플로/언더플로 panic(0x11)이다", async function () {
  const counter =
    await ethers.deployContract("Counter");

  await expect(
    counter.subtract(1n)
  ).to.be.revertedWithPanic(0x11);    // 실패 이유까지 검사
});
```

```
  ✔ 0에서 1을 빼면 revert된다
  ✔ revert 이유는 산술 오버플로/언더플로 panic(0x11)이다

6 passing (57ms)
```

- ✓ **Solidity 0.8 이상: 범위를 벗어난 계산은 자동으로 revert (Panic 0x11)**

참고 – Ignition

❑ 예: 배포 + 함수 호출 + 값 읽기 + ETH 전송을 한 모듈로

```
// ignition/modules/CounterSetup.ts
export default buildModule("CounterSetupModule", (m) => {
  const counter = m.contract("Counter");           // ① 배포

  const add5 = m.call(counter, "add", [5n]);       // ② add(5) 호출 (트랜잭션)

  m.staticCall(counter, "count", [], 0,            // ③ count() 읽기 (트랜잭션 X)
    { after: [add5] });

  m.send("sendEth", m.getAccount(1), 10n ** 15n,   // ④ 계정 1에게 0.001 ETH
    undefined, { after: [add5] });

  return { counter };
});
```

```
PS> npx hardhat ignition deploy ignition/modules/CounterSetup.ts --network localhost
Batch #1   Executed CounterSetupModule#Counter
Batch #2   Executed CounterSetupModule#Counter.add
Batch #3   Executed CounterSetupModule#Counter.count
           Executed CounterSetupModule#sendEth
[ CounterSetupModule ] successfully deployed 🚀
```

- ✓ 결과 확인: count = 5n, 계정 1 잔액 10000 → 10000.001 ETH
- ✓ Ignition이 순서를 정해 Batch로 실행 → 같은 Batch는 동시에 실행되므로 순서가 필요하면 `after`로 지정

참고 - Ignition

❑ 예: 이미 배포된 컨트랙트에 연결해서 호출 (m.contractAt)

```
// ignition/modules/CounterAt.ts
export default buildModule("CounterAtModule", (m) => {
  const counter = m.contractAt(                    // 새로 배포하지 않고 연결
    "Counter",
    "0x5FbDB2315678afecb367f032d93F642f64180aa3"   // deployed_addresses.json의 주소
  );
  m.call(counter, "increment");                    // increment() 호출
  return { counter };
});
```

```
PS> npx hardhat ignition deploy ignition/modules/CounterAt.ts --network localhost
  Executed CounterAtModule#Counter
  Executed CounterAtModule#Counter.increment
[ CounterAtModule ] successfully deployed 🚀
→ count: 5n → 6n
```

❑ 주의 (실제 실행으로 확인)

- ✓ **m.contractAt은 주소에 컨트랙트가 있는지 확인하지 않음 → 틀린 주소도 성공처럼 보임**

Q&A